

84. Largest Rectangle in Histogram **Hard**

Acceptance: 49.1% | [LeetCode](#)

Given an array of integers `heights` representing the histogram's bar height where the width of each bar is `1`, return *the area of the largest rectangle in the histogram*.

Example 1:

```
**Input:** heights = [2,1,5,6,2,3]
**Output:** 10
**Explanation:** The above is a histogram where width of each bar is 1.
```

The largest rectangle is shown in the red area, which has an area = 10 units.

Example 2:

```
**Input:** heights = [2,4]
**Output:** 4
```

Constraints:

- `1 <= heights.length <= 105`
- `0 <= heights[i] <= 104`

Claude Opus Solution

Largest Rectangle in Histogram

1. Intuition

The **key insight**: For every bar, the largest rectangle that uses that bar as its **shortest height** extends as far left and right as possible until it hits a bar that is *shorter* than it.

So for each bar `i` with height `h`, we want to find: - **Left boundary**: the first bar to the left that is shorter than `h` - **Right boundary**: the first bar to the right that is shorter than `h`

The rectangle width is then `(right - left - 1)`, and its area is `h * width`.

The **monotonic increasing stack** is the perfect tool here. As we iterate, we keep bars in increasing height order in the stack. When we encounter a bar *shorter* than the top of the stack, that bar marks the **right boundary** for the popped bar, and the new stack top marks its **left boundary**. This lets us compute each bar's maximal rectangle in $O(1)$ when it's popped.

2. Approach (Pseudocode)

- Initialize stack with sentinel index `-1` (acts as the left boundary "wall")
- Append a sentinel height of `-1` (or `0`) to the heights to force flush the stack at the end
- For each (index, height):
 - While stack top's height > current height:
 - Pop it; this popped bar's right boundary is `index`
 - Its left boundary is the new stack top
 - `width = index - stack_top - 1`
 - `area = popped_height * width`
 - update max area
 - Push current index onto the stack
- Return the maximum area found

3. Python Solution

```
from typing import List
```

```
class Solution:
```

```
    def largestRectangleArea(self, heights: List[int]) -> int:
        # Stack holds INDICES of bars, kept in increasing-height order.
        # We seed it with -1 to serve as a left-wall sentinel so that
        # width calculation works even when a bar extends to the far left.
        stack = [-1]
        largest_area = 0

        # Append a sentinel bar of height -1 at the end.
        # Since -1 is smaller than any real height (heights[i] >= 0),
        # this forces every remaining bar in the stack to be popped
        # and processed, avoiding a separate cleanup loop after.
        heights.append(-1)
```

```

for index, height in enumerate(heights):
    # Maintain increasing stack: while the bar on top is TALLER
    # than the current bar, the current bar is its right boundary.
    while stack[-1] != -1 and heights[stack[-1]] > height:
        # Pop the taller bar -> this bar is the rectangle's height.
        current_height = heights[stack.pop()]

        # Left boundary is now the new stack top (a shorter bar
        # or the -1 sentinel). Right boundary is `index`.
        # Width = (right - left - 1) gives count of contiguous bars
        # all >= current_height.
        current_width = index - stack[-1] - 1

        # Area for the rectangle using current_height as the min.
        current_area = current_height * current_width
        largest_area = max(current_area, largest_area)

    # Push current index; the stack stays in increasing-height order.
    stack.append(index)

# Clean up the sentinel we added (good practice to not mutate input
# permanently). Optional if the caller doesn't reuse the list.
heights.pop()

return largest_area

```

Walkthrough on [2,1,5,6,2,3]

After appending sentinel `-1`: `[2,1,5,6,2,3,-1]` - Process 2: push. Stack: `[-1, 0]` - Process 1: pop 2 → width `1-(-1)-1=1`, area 2. Push 1. Stack: `[-1, 1]` - Process 5, 6: push. Stack: `[-1,1,2,3]` - Process 2: pop 6 → area `6*1=6`; pop 5 → width `4-1-1=2`, area `5*2=10` ✓. Push. Stack: `[-1,1,4]` - Process 3: push. Stack: `[-1,1,4,5]` - Process `-1`: pop everything; max stays 10.

4. Complexity Analysis

- **Time: $O(n)$** — Each index is **pushed exactly once** and **popped at most once**. Although there's a nested `while` loop, the total number of push/pop operations across the entire run is bounded by $2n$, giving amortized $O(1)$ work per element.
- **Space: $O(n)$** — In the worst case (strictly increasing heights like `[1,2,3,4,5]`), all indices sit in the stack simultaneously before being flushed by the sentinel.

5. Alternative Approaches

Approach	Time	Space	Notes
Brute force (all pairs)	$O(n^2)$	$O(1)$	For each pair of bars, find min height and compute area. Too slow for $n=10^5$.
Divide & Conquer	$O(n \log n)$ avg, $O(n^2)$ worst	$O(n)$	Find the min bar; the answer is <code>max(left_half, right_half, full_width_at_min)</code> . Degrades to $O(n^2)$ on sorted input.
Precompute left/right boundaries (two passes)	$O(n)$	$O(n)$	Use two arrays storing the index of the previous-smaller and next-smaller element for each bar, then compute <code>height * (right[i]-left[i]-1)</code> . Same complexity, more explicit but uses more arrays.
Monotonic Stack (this solution)	$O(n)$	$O(n)$	Cleanest optimal approach; single pass.

Key pattern to remember: Whenever a problem asks for "the largest/smallest area or value where each element must consider the *nearest smaller/greater element* on its sides," a **monotonic stack** is almost always the optimal tool. This same technique applies to problems like *Trapping Rain Water*, *Maximal Rectangle (in a 2D matrix)*, and *Sum of Subarray Minimums*.

Codex Solution

1. Intuition

For every bar `heights[i]`, imagine using it as the **shortest bar** of some rectangle.

If `heights[i]` is the shortest bar, then the rectangle can extend:

- left until we find a bar shorter than `heights[i]`
- right until we find a bar shorter than `heights[i]`

So the key question is:

For each bar, how far can it extend while still being the minimum height?

A monotonic increasing stack helps us answer this efficiently.

We maintain a stack of indices whose heights are in **non-decreasing order**.

When we see a bar shorter than the top of the stack, it means:

The current bar is the first smaller bar to the right of the stack top.

So we can pop the taller bar and calculate the largest rectangle where that popped bar is the minimum height.

2. Approach

Key idea

Use a stack to store indices of bars in increasing height order.

When `heights[i]` is smaller than the height at the top of the stack, pop from the stack and calculate area.

For a popped index `top`:

```
height = heights[top]
right boundary = i - 1
left boundary = stack[-1] + 1
width = i - stack[-1] - 1
area = height * width
```

We use a sentinel index `-1` to simplify width calculation.

We also process one extra artificial bar of height `0` at the end to flush all remaining bars from the stack.

Pseudocode

```
largestRectangleArea(heights):
    stack = [-1]
    max_area = 0

    for i from 0 to len(heights):
        if i == len(heights):
            current_height = 0
        else:
            current_height = heights[i]

        while stack top is not -1 and heights[stack top] > current_height:
            popped_index = stack.pop()
            height = heights[popped_index]
            width = i - stack[-1] - 1
            area = height * width
            max_area = max(max_area, area)

        push i onto stack

    return max_area
```

3. Python Solution

```
from typing import List

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        """
        Return the area of the largest rectangle in the histogram.

        We use a monotonic increasing stack of indices.
        The heights corresponding to indices in the stack are non-decreasing.
        """

        n = len(heights)

        # Stack stores indices, not heights.
        # The sentinel -1 helps calculate widths cleanly.
        stack = [-1]

        max_area = 0

        # We iterate one step beyond the array.
        # At i == n, we use a fake height 0 to force all remaining bars to pop.
        for i in range(n + 1):

            # Fake trailing bar of height 0 to flush the stack.
            current_height = 0 if i == n else heights[i]

            # If current height is smaller than the height at stack top,
            # then the bar at stack top cannot extend further right.
            while stack[-1] != -1 and heights[stack[-1]] > current_height:

                # This popped bar is the limiting height of a rectangle.
                height = heights[stack.pop()]

                # After popping, stack[-1] is the index of the previous
                # smaller bar to the left.
                #
                # Current index i is the first smaller bar to the right.
                #
                # Therefore, the rectangle spans:
                #   from stack[-1] + 1 to i - 1
                width = i - stack[-1] - 1

                # Compute rectangle area using popped height as minimum height.
                area = height * width

                # Update best answer.
                max_area = max(max_area, area)

            # Push current index.
            # The stack remains monotonic increasing by height.
            stack.append(i)

        return max_area
```

4. Complexity Analysis

Let `n = len(heights)`.

Time Complexity

$O(n)$

Each index is pushed onto the stack once and popped from the stack once.

Even though there is a nested `while` loop, the total number of stack operations across the whole algorithm is linear.

Space Complexity

```
O(n)
```

In the worst case, the stack can store all indices.

For example:

```
heights = [1, 2, 3, 4, 5]
```

All bars are pushed before any are popped.

5. Alternative Approaches

1. Brute Force

For every possible subarray, compute the minimum height and area.

```
Time: O(n^3)
Space: O(1)
```

Can be improved slightly by maintaining the minimum height while expanding the right boundary.

```
Time: O(n^2)
Space: O(1)
```

Too slow for $n \leq 10^5$.

2. Precompute Previous Smaller and Next Smaller

For each index, compute:

- index of previous smaller element
- index of next smaller element

Then for each bar:

```
width = next_smaller[i] - previous_smaller[i] - 1
area = heights[i] * width
```

This also uses monotonic stacks.

```
Time: O(n)
Space: O(n)
```

The single-stack solution above is essentially a more space-efficient version of this idea because it calculates areas as soon as the right boundary is known.

3. Divide and Conquer

Choose the minimum height bar in a range as the limiting height.

For a range `[left, right]`, the largest rectangle is the maximum of:

1. largest rectangle on the left side
2. largest rectangle on the right side
3. rectangle spanning the whole range using the minimum height

Naive divide and conquer takes:

```
Time: O(n^2)
```

With a segment tree or RMQ structure to find minimum indices efficiently:

```
Time: O(n log n)
Space: O(n)
```

Still more complex than the monotonic stack solution.